

Sky Analytics Stream

Architettura Distribuita per l'Analisi in Tempo Reale del Traffico Aereo

Flavio Simonelli

Matricola 0365333

Dipartimento di Ingegneria

Università di Roma "Tor Vergata"

flavio.simonelli@alumni.uniroma2.eu

Francesco Masci

Matricola 0365258

Dipartimento di Ingegneria

Università di Roma "Tor Vergata"

francesco.masci.02@alumni.uniroma2.eu

Sommario—Questo lavoro presenta *Sky Analytics Stream*, un'architettura distribuita ed elastica per l'analisi e il monitoraggio in tempo reale del traffico aereo statunitense basata su Apache Flink e Apache Kafka. Il sistema riceve i dati da un simulatore ad alte prestazioni sviluppato in Go, in grado di riprodurre flussi di eventi storici in event-time con disordine controllato e fattori di accelerazione variabili. Attraverso un'unica topologia a grafo diretto aciclico (DAG), la pipeline esegue in parallelo tre analisi: aggregazioni orarie delle performance delle compagnie aeree, classifiche in tempo reale (Top-10) degli aeroporti con ritardi severi e stima dei percentili dei ritardi tramite l'algoritmo DDSketch. Risultati e metriche di telemetria sono storicizzati in InfluxDB e visualizzati tramite Grafana. La tolleranza ai guasti è garantita da checkpointing *exactly-once* con stato incrementale gestito da RocksDB. Il sistema, testato sia localmente che su cluster Amazon EC2, sostiene throughput superiori a 9.000 eventi/secondo. La valutazione sperimentale analizza le prestazioni del sistema al variare del parallelismo, del disordine degli eventi e della frequenza dei checkpoint.

Index Terms—Apache Flink, Stream Processing, Apache Kafka, Traffico Aereo, Percentili Approssimati, Tolleranza ai Guasti, DDSketch, checkpointing.

I. INTRODUZIONE

L'analisi tempestiva del traffico aereo richiede di elaborare flussi continui di dati per aggiornare metriche prestazionali e classifiche di ritardo in tempo reale. In questo contesto, l'accuratezza e la freschezza dei risultati dipendono dalla capacità di gestire le sfide intrinseche dello stream processing. In particolare, ritardi di rete, colli di bottiglia nell'ingestione e arrivi fuori ordine fanno sì che il tempo logico in cui un volo si verifica (*event-time*) non venga rispettato nell'ordine di come gli eventi arrivano ai processori di elaborazione. Per fornire risultati coerenti e completi, è dunque cruciale definire politiche di watermarking e di gestione dei dati tardivi che bilancino correttamente la latenza e l'accuratezza delle aggregazioni.

Per affrontare tali sfide, questo lavoro presenta il progetto e la valutazione sperimentale di *Sky Analytics Stream*, un'architettura distribuita progettata per elaborare un dataset storico di circa 2,2 milioni di voli negli Stati Uniti (relativi al periodo gennaio–aprile 2025). Il dataset viene riprodotto in modo controllato tramite un simulatore ad alte prestazioni sviluppato in Go, in grado di iniettare dati in Apache Kafka applicando fattori di compressione temporale e introducendo disordine artificiale. La pipeline di elaborazione principale è

realizzata in Apache Flink e implementa un singolo grafo aciclico (DAG) ottimizzato per eseguire in parallelo tre analisi distinte:

- 1) Il calcolo orario delle statistiche di volo per ciascuna compagnia aerea;
- 2) Il tracciamento dinamico delle classifiche (Top-10) degli aeroporti con i maggiori ritardi accumulati;
- 3) La stima continua della distribuzione dei ritardi complessivi.

Il resto della relazione è organizzato come segue: la Sezione II descrive l'architettura generale del sistema e il flusso dei dati; la Sezione III approfondisce i dettagli implementativi delle singole componenti; la Sezione IV illustra l'infrastruttura di deployment; la Sezione V presenta la valutazione sperimentale e i risultati dei test prestazionali; infine, la Sezione VI conclude il report delineando possibili sviluppi futuri.

II. ARCHITETTURA DEL SISTEMA

III. IMPLEMENTAZIONE DELLE COMPONENTI

A. Simulatore di Ingestione in Go

L'ingestione e il replay degli eventi storici in tempo reale sono gestiti da un componente custom sviluppato in linguaggio **Go**. La scelta tecnologica è motivata dalla necessità di disporre di un agente di ingestione ad altissime prestazioni, caratterizzato da un consumo minimo di risorse e in grado di garantire throughput elevati (fino a oltre 9.000 eventi/secondo) senza introdurre colli di bottiglia o latenze spurie dovute alla Garbage Collection (frequenti in ecosistemi basati su JVM).

Il simulatore implementa diverse funzionalità avanzate e ottimizzazioni ingegneristiche:

- **Ingestione Streaming Zero-Copy ed Integrity Check:** Al primo avvio, l'applicazione verifica l'integrità del dataset scaricato confrontando l'hash SHA-1 calcolato con quello di riferimento. L'archivio `.tar.gz` originale viene letto e decodificato al volo in memoria (*on-the-fly streaming*) tramite la libreria nativa di Go, senza la necessità di estrarre preventivamente su disco circa 2.2 milioni di righe in formato CSV, risparmiando spazio di archiviazione ed eliminando i tempi morti di I/O del disco.
- **Binary Caching in formato GOB:** Al fine di minimizzare i tempi di inizializzazione nelle successive

esecuzioni (stress test o tuning della pipeline), gli eventi letti vengono convertiti in strutture tipizzate Go, ordinati cronologicamente rispetto all'event-time calcolato e serializzati in un file di cache binario locale in formato **GOB** (Go Object Binary). Questo meccanismo riduce il tempo di boot e caricamento del dataset da circa 90 secondi reali a meno di 2 secondi.

- **Gestione del Tempo Fisico ed Event-Time Replay:** Per ciascun volo, l'event-time logico viene ricostruito dinamicamente combinando i campi YEAR, MONTH, DAY_OF_MONTH e l'orario programmato CRS_DEP_TIME. Il replay emula la distanza temporale reale tra i voli applicando un fattore di compressione del tempo (*Speedup Factor*) impostabile da configurazione. La temporizzazione è calcolata tramite un meccanismo a precisione millesimale che valuta la differenza relativa (Δt) dell'event-time rispetto all'evento precedente, attendendo il tempo residuo riscalato prima dell'invio a Kafka.
- **High-Precision Hybrid Waiter (Sleep-Spinlock):** Per riprodurre fedelmente la distanza temporale tra gli eventi, specialmente con alti fattori di accelerazione (dove le finestre temporali logiche di un'ora si contraggono in pochi millisecondi reali), il simulatore non può affidarsi unicamente alle funzioni di *sleep* fornite dal sistema operativo. Queste ultime sono infatti soggette all'imprecisione dello scheduler del kernel e all'overhead dei *context switch*, che introducono latenze variabili nell'ordine dei 10-15 ms. Per ovviare a questo limite, è stata implementata una strategia di attesa ibrida. Se la durata dell'attesa supera una determinata soglia critica (*spin threshold*, impostata a 15 ms), il thread esegue una sleep cooperativa rilasciando la CPU; non appena il tempo residuo scende sotto la soglia, il thread effettua un **spinlock** (attesa attiva ad alta frequenza). Questo impedisce al sistema operativo di sospendere il processo, assicurando una precisione di rilascio nell'ordine dei microsecondi e garantendo l'uniformità del throughput.
- **Generazione di Disordine (Out-of-Order Injection):** Al fine di stressare e validare le strategie di Watermarking e la gestione dei dati tardivi (*allowed lateness*) di Apache Flink, il simulatore implementa un decoratore basato su un algoritmo probabilistico. Con una probabilità definita $P \in [0, 1]$, un record viene selezionato e il suo istante di pubblicazione viene dislocato in avanti nel tempo di un offset casuale limitato da un valore massimo configurabile (δ_{max} in minuti logici). La sequenza di eventi risultante viene infine ri-ordinata sul tempo di pubblicazione reale prima dell'invio, riproducendo accuratamente i tipici disallineamenti di rete di uno stream distribuito.

IV. INFRASTRUTTURA DI DEPLOYMENT

V. VALUTAZIONE SPERIMENTALE

VI. CONCLUSIONI E SVILUPPI FUTURI

RIFERIMENTI BIBLIOGRAFICI

- [1] L. Fernando, H. Bindra, and K. Daudjee, "An experimental analysis of quantile sketches over data streams," in *Proceedings of the 26th International Conference on Extending Database Technology (EDBT '23)*, pp. 1–12, 2023.
- [2] C. Masson, J. E. Rim, and H. K. Lee, "DDSketch: A fast and fully-mergeable quantile sketch with relative-error guarantees," *Proceedings of the VLDB Endowment*, vol. 12, no. 12, pp. 2195–2205, 2019.

APPENDICE A
UTILIZZO DI STRUMENTI DI INTELLIGENZA ARTIFICIALE GENERATIVA

APPENDICE B
RAPPRESENTAZIONI GRAFICHE E ALLEGATI